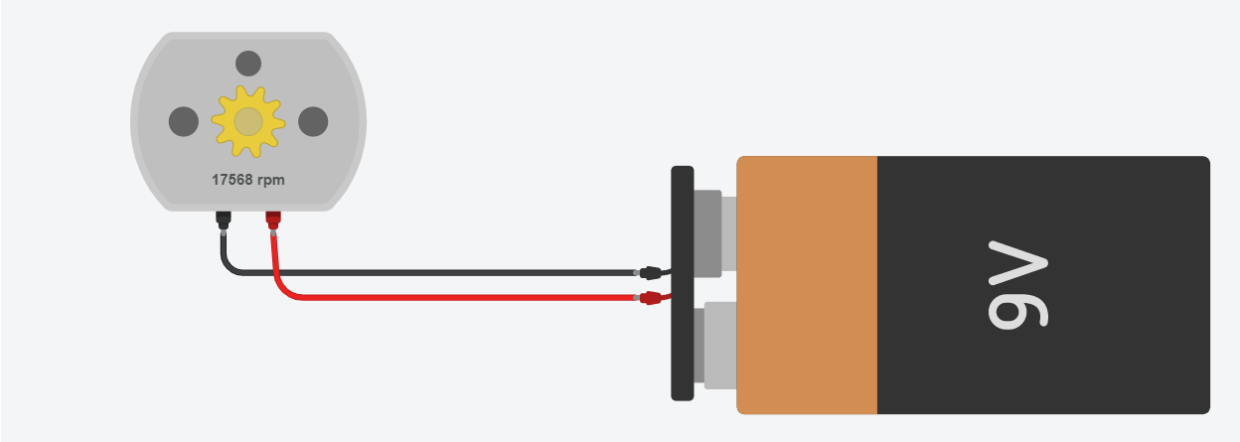
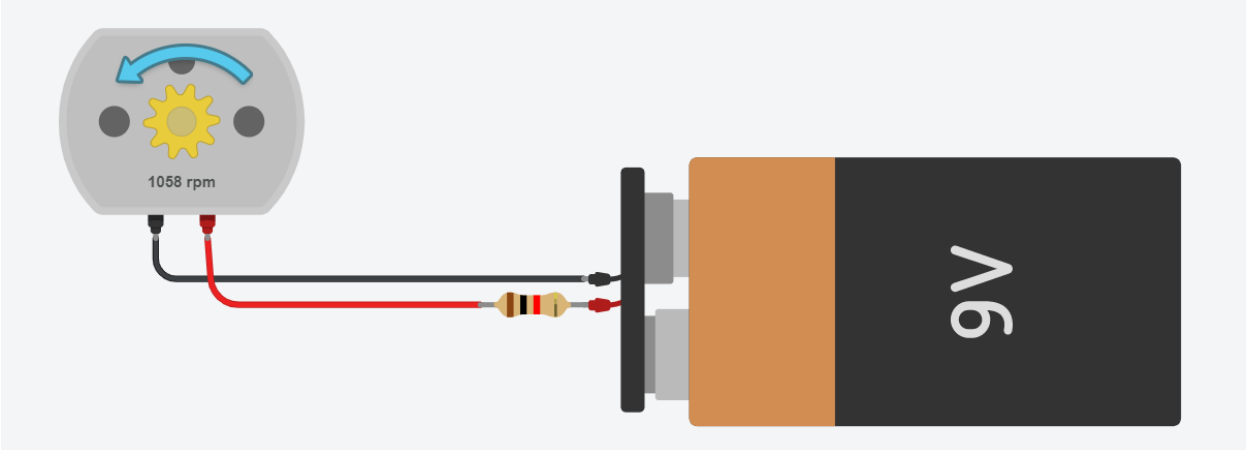
# Transistors and DC Motor Control

In this exercise, we'll look at basic DC motor operation, as well as how simple electronic control of them can be obtained with a transistor.

Begin by bring in a power source (9v Battery) and a DC motor into the TinkerCAD project space, and wire the two terminals of the battery to the two terminals of the motor. *Match red to red and black to black.* Once complete, start the simulation.

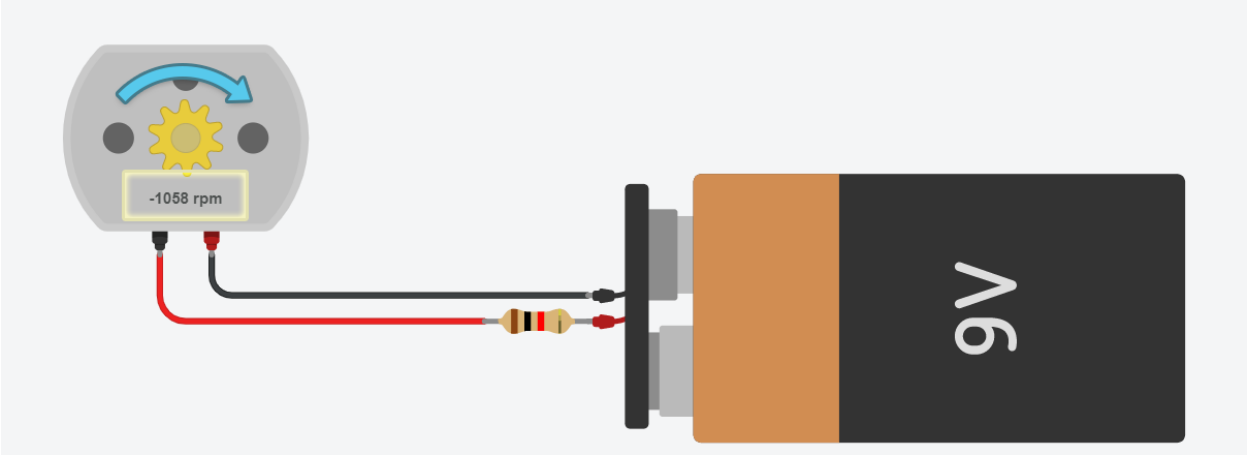



You should see the motor spinning, although it is spinning quite fast (>17000 rpm), and it is difficult to tell which direction the motor is even rotating! Let's add a 1k Ω into the circuit and see what changes when the simulation is run:

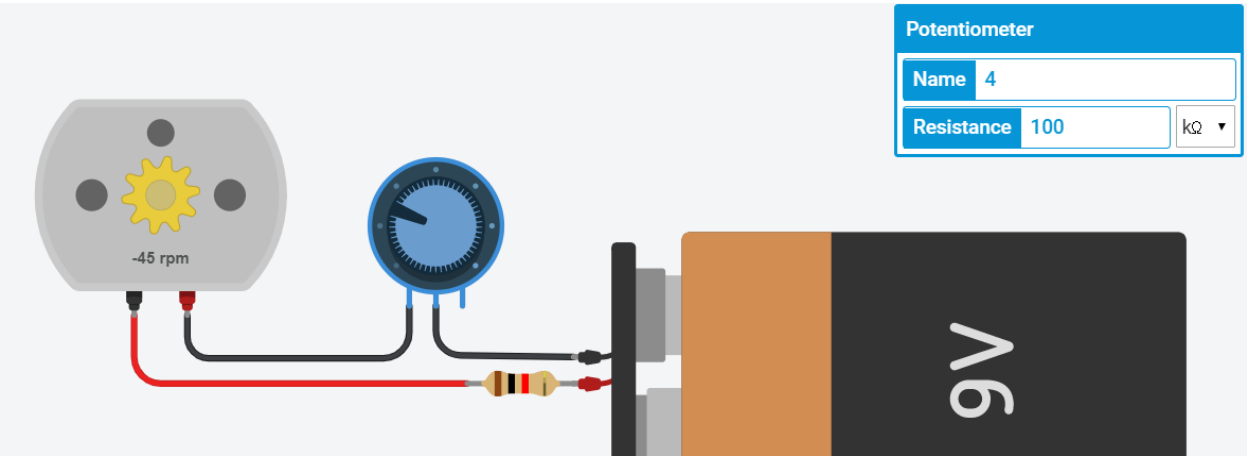



The motor runs **much slower** (around 1000 rpm). The speed that a DC motor turns is directly related to the voltage applied to the motor. DC motors will be rated for a particular voltage range (one example could be 3-6V). Supplying a lower voltage will mean the *motor will not turn*, and supplying a higher voltage will usually mean *the motor spins even faster, but will have a shorter lifespan before burning out.* If the voltage is high enough this will happen very quickly.

Next, let's try **reversing the polarity** of this circuit, so that the black terminal of the battery connects to the red terminal of the motor and the red battery terminal to the black motor terminal. When you run the simulation we should see the motor spinning *at the same speed, but in the opposite direction.* (note the negative RPM value representing *clockwise rotation in TinkerCAD).* This is an important property of motors. **Amount of voltage controls the speed of the motor** and **polarity of connection determines the direction of rotation.**

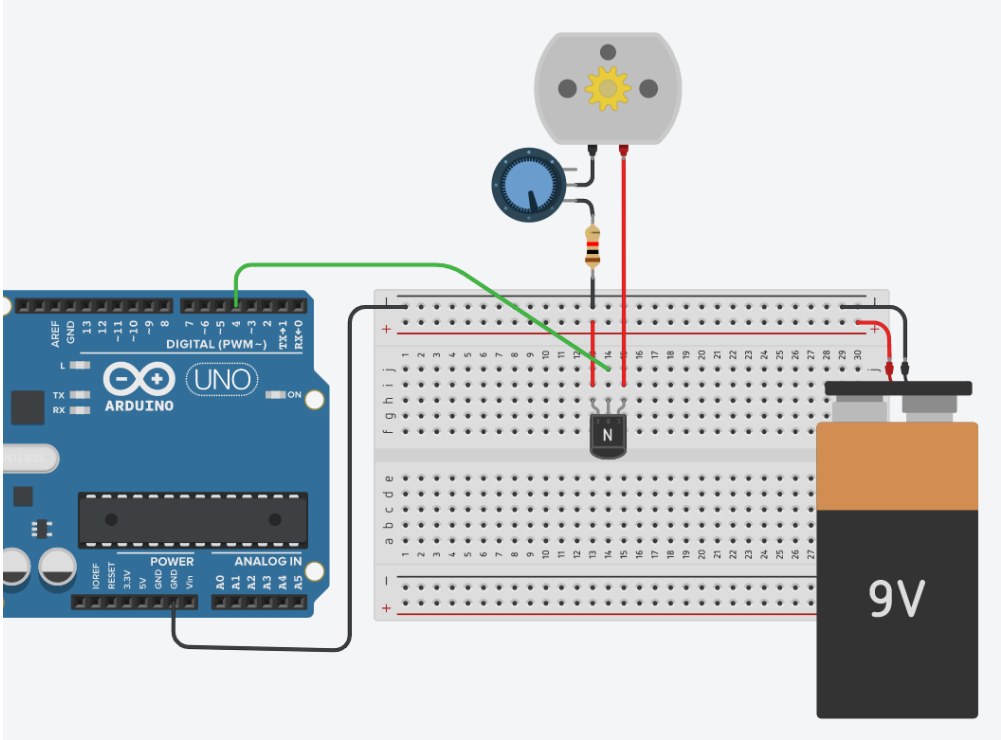



Lastly, let's add a potentiometer (set to 100 KΩ) so that we have some mechanical control over the motor speed.

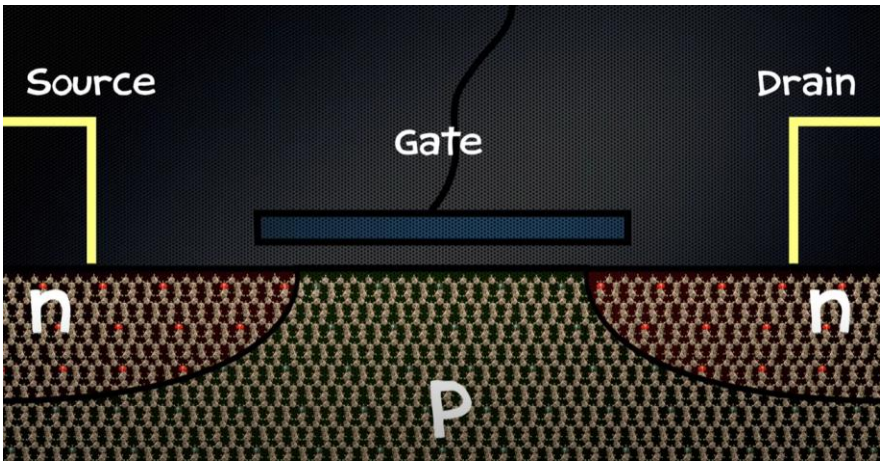


**Transistor Control**

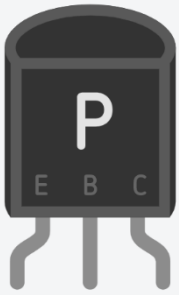
Controlling a motor mechanically (with an on/off switch) would is the simplest and least flexible way to control a motor. There are many different approaches to gaining fine control over a DC motor speed and direction (H-Bridge, Motor Controller board), but for this exercise we'll look at **using an NPN transistor as a switch.** Begin by adding an **Arduino, Breadboard,** and **NPN transistor** to your TinkerCAD project space. Re-arrange the components to match the circuit diagram below:




Notice how in this circuit, the 9V battery source connects to the DC motor across the **source** and **drain** pins of the transistor. Electrons can only move between these two parts of a transistor when there is a positive charge present on the middle (**gate**) pin. Since we have a wire connecting the **gate** to **digital pin 4,** we should be able to allow or restrict current flow to the motor by varying if pin 4 is outputting a HIGH or LOW voltage through code.




**Note:** There are two main types of transistors (BJT and MOSFET). For this exercise we'll use BJT, but you may notice the labels are slightly different than listed on the left.

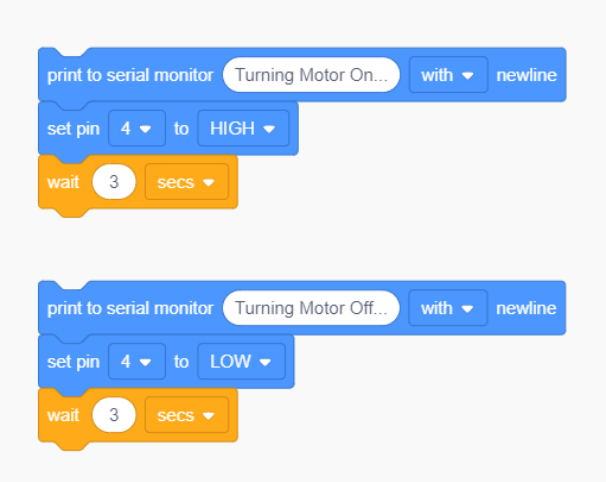



The labels, although named differently, still represent the same transistor components:

| | | |
|---|---|---|
| **Collector** | is analogous to | **Source** |
| **Base** | is analogous to | **Gate** |
| **Emitter** | is analogous to | **Drain** |

Adding the Code:

Basic on/off control of the DC motor requires only a simple program that alternates HIGH/LOW on the digital pin connected to the **gate.** Here is code in both **block and text form:**




```
void setup()
{
  pinMode(3, OUTPUT);
}

void loop()
{
  digitalWrite(3, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(3, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

**PIR Sensor**



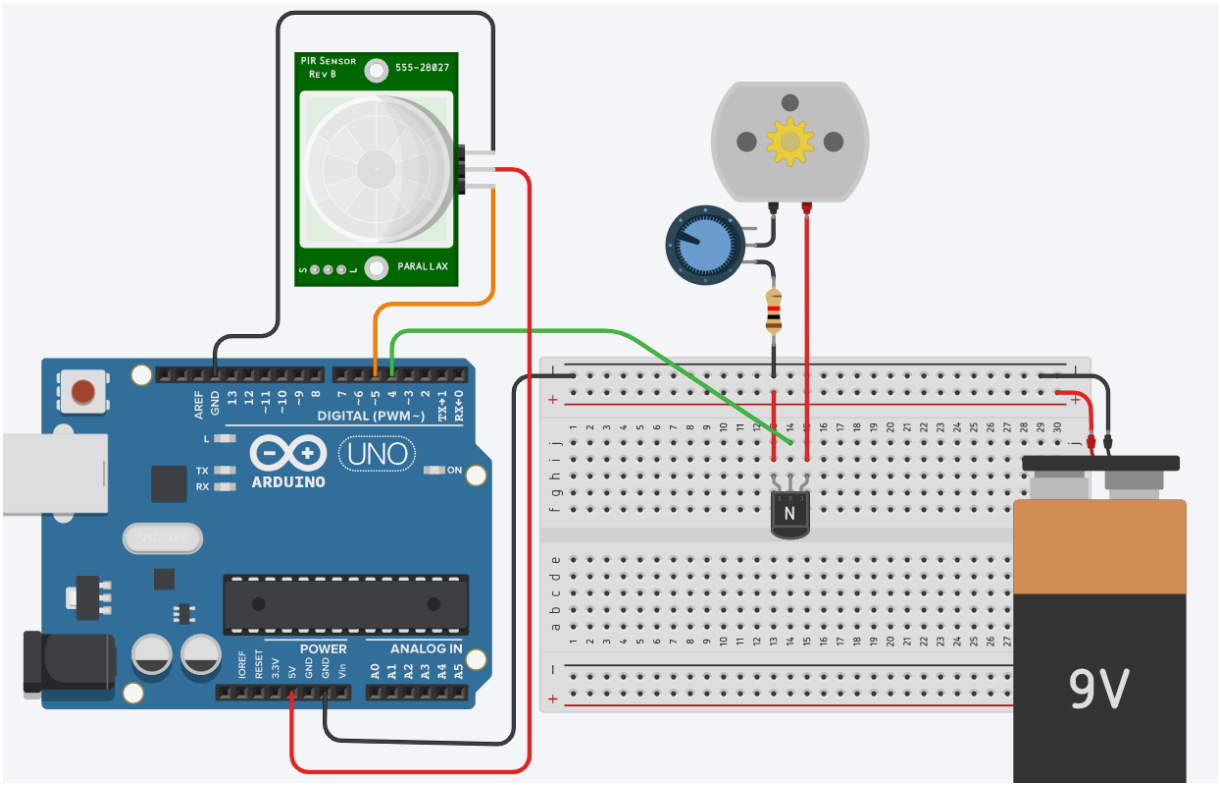
A **Passive InfraRed (PIR)** sensor is a component that can be used to sense human motion with good reliability in the detection range (typically up to 20 feet in a 70-110° angle). Their ease-of-use and fairly rugged construction make them quite popular in home and business electronics.

Search in the “All Components” section to find and add a PIR sensor to our project space.




Before we wire it into the circuit, remember that right now the power rails on the breadboard carry 9V rather than 5V. All electrical components have **datasheets** that provide specifications like acceptable input voltage ranges. Although many PIR sensors can accept a range of voltages, the one being simulated in TinkerCAD needs 5V. *Giving it a higher voltage will prevent it from working correctly!*

So let's wire the PIR sensor directly to the Arduino as is shown below. You don't need to connect it to the breadboard at all.

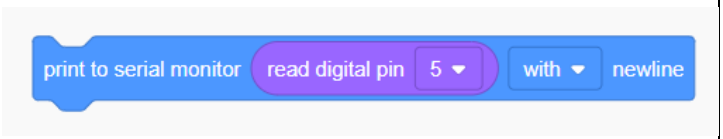



**Testing and Understanding the Readings**

We've connected 3 pins to this sensor. Two are dedicated to 5v and GND, and the last is a **signal** wire, which we've connected to **digital pin 5.** To read the sensor, we should do a **digitalRead()** on pin 5. A good way to test this is to use a *small program that prints this reading to the Serial Monitor.*

- - Comment out (if using text code) or delete (if using blocks) your previous code. We'll need it back in a bit, but for now don't want to run it anymore. - -

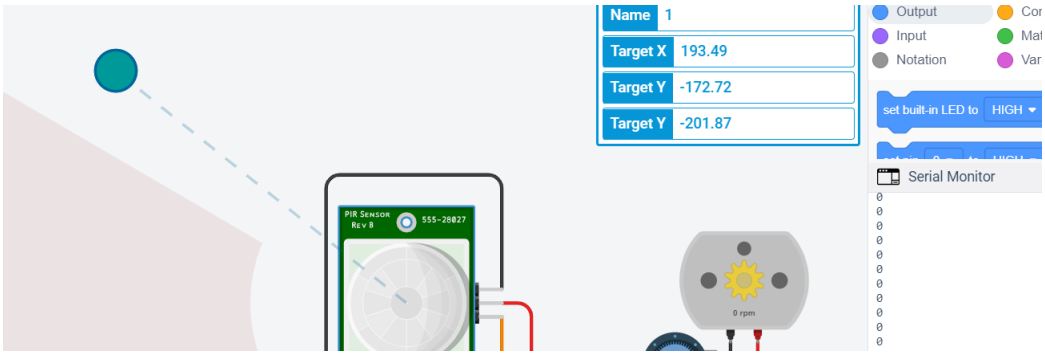
Instead, code in the following short program:



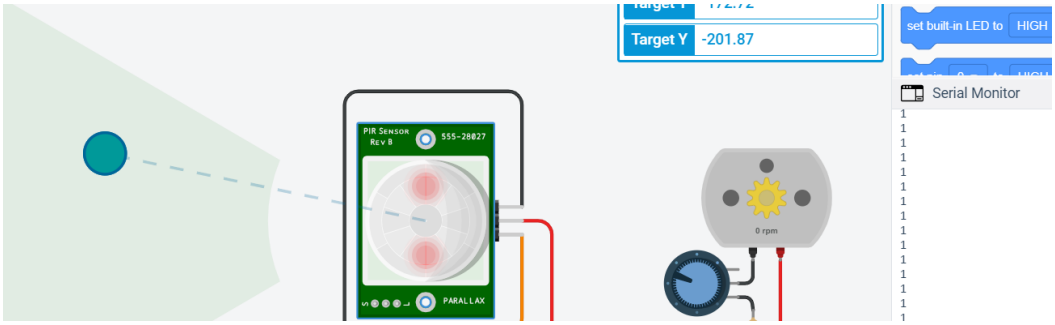

```
void setup()
{
  pinMode(5, INPUT);
  Serial.begin(9600);
}

void loop()
{
  Serial.println(digitalRead(5));
  delay(10); // Delay for simulation
}
```

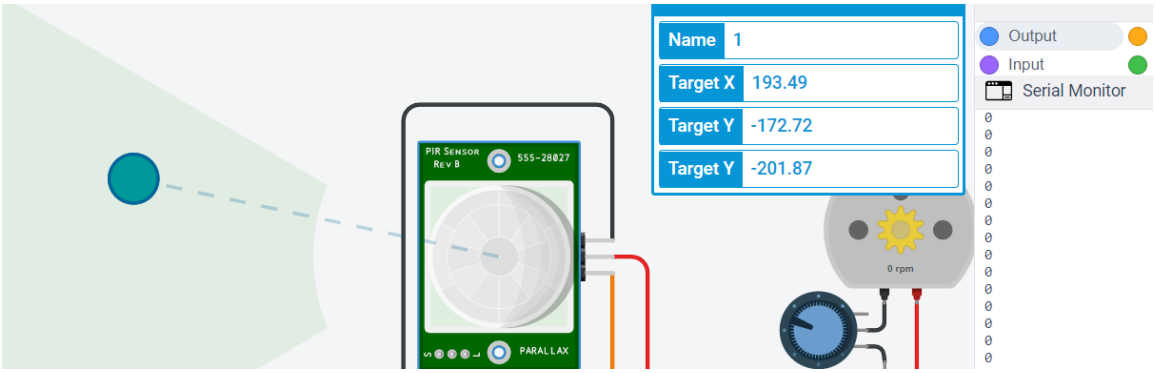
Start the simulation and open the Serial Monitor. You should see all 0s being printed out. This means "no motion detected". If you click on the PIR sensor while the simulation is running, you'll be given an object that you can control and the sensor range will be illustrated. When that object (rendered as an ellipse) is outside of the range, we'll see continual readings of 0.




When the object is moved into the range, however, we should see the Serial Monitor begin printing out 1s. This means "Motion is detected".




However, if you leave the object inside the detector range for a few seconds without moving it, you'll see the readings go back to 0 ("no motion detected"). This type of sensor doesn't detect objects themselves, but the **motion of objects**, so it is possible for a stationary or very slowly moving object to not necessarily be detected!

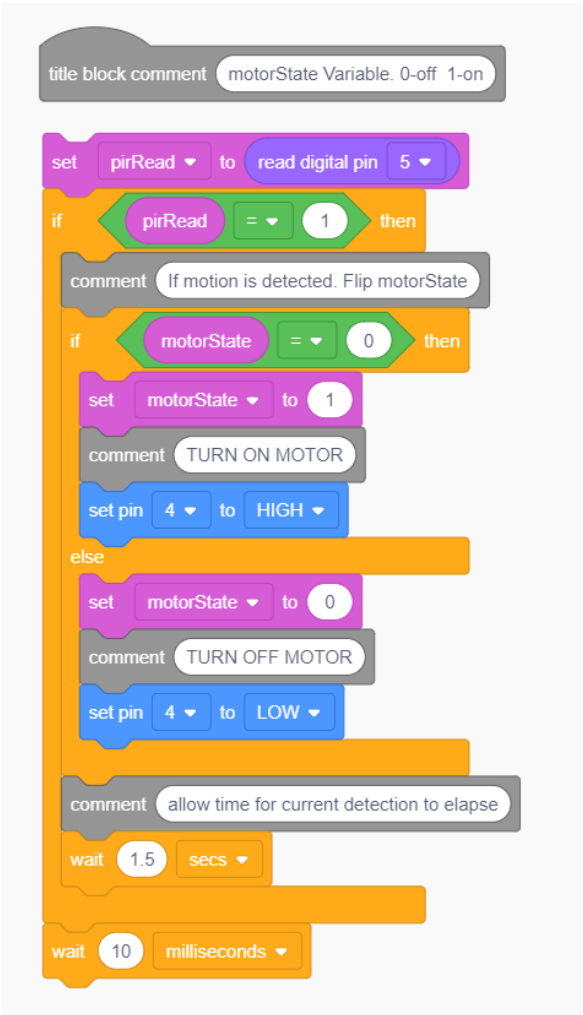


**Motion-Activated Motor Control**

Let's merge the two programs we have created:

1. DC Motor on/off control with transistor
2. PIR Motion detector

This simple program will have the **motion being detected by the PIR sensor act as a switch.** Whenever motion is detected, turn the DC motor on or off, whichever state is not currently active. *Code on how to do this will be provided below, but try to write this code on your own first!*




```
/*
motorState Variable. 0-off  1-on
*/

#define motorPin 4
#define pirPin 5

int motorState = 0;
int pirRead = 0;

void setup()
{
  pinMode(pirPin, INPUT);
  pinMode(motorPin, OUTPUT);
}

void loop()
{
  pirRead = digitalRead(pirPin);
  if (pirRead == 1) {
    // If motion is detected. Flip motorState
    if (motorState == 0) {
      motorState = 1;
      // TURN ON MOTOR
      digitalWrite(motorPin, HIGH);
    } else {
      motorState = 0;
      // TURN OFF MOTOR
      digitalWrite(motorPin, LOW);
    }
    // allow time for current detection to elapse
    delay(1500); // Wait for 1500 millisecond(s)
  }
  delay(10); // Wait for 10 millisecond(s)
}
```